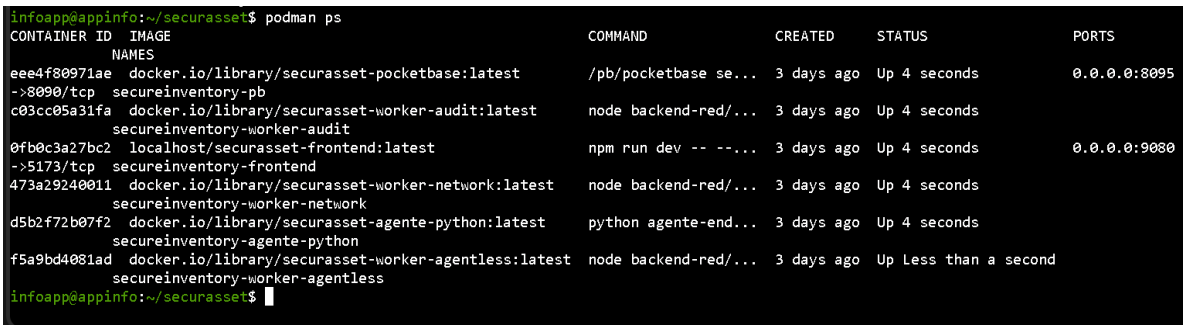
Resumen de Arquitectura y Flujo de Datos

El sistema SecurAsset utiliza una arquitectura distribuida basada en microservicios containerizados mediante Podman / Docker. La solución está dividida de forma modular en tres capas principales que interactúan entre sí de forma asíncrona:

- **Capa de Persistencia (Backend-as-a-Service - PocketBase):** Expuesta en el puerto 8095, actúa como el núcleo central de base de datos relacional y servidor de eventos en tiempo real (*Realtime Server Sent Events - SSE*). Es la encargada de almacenar los registros de activos, evaluaciones de seguridad, auditorías de red y tablas de cuarentena (*Shadow IT*).
- **Capa Frontend (SvelteKit & Svelte 5):** Interfaz web corporativa expuesta en el puerto 9080. Es un tablero de control SPA (*Single Page Application*) altamente reactivo que consume la API REST de PocketBase y se suscribe a los cambios en tiempo real mediante *sockets* HTTP.

- **Capa de Motor de Auditoría y Agentes (Node.js & Python):**
    - **worker-network.js:** Servicio en Node.js que escanea periódicamente la subred mediante sondeos ARP/Nmap, clasifica dispositivos según su prefijo MAC OUI y detecta equipos no autorizados enviándolos a cuarentena.
    - **worker-audit.js:** Servicio en Node.js enfocado en auditar la seguridad de servidores y NVRs (verificando puertos abiertos, firmas HTTP/TLS y contraseñas por defecto).
    - **agente.py:** Ejecutable de inspección instalado en las estaciones de trabajo que recolecta métricas internas (BitLocker/LUKS, EDR, parches del sistema operativo y lista de administradores) y las envía a PocketBase.



Módulo Frontend (Svelte 5 & SvelteKit) – App.svelte

Inicialización y Cliente PocketBase

Importaciones e Inicialización del SDK de PocketBase

El archivo App.svelte actúa como el punto de entrada y orquestador principal de toda la aplicación web. En su bloque inicial de script, importa todas las vistas secundarias, modales de gestión, navegadores y alertas tipo *Toast*.

Para conectarse con la base de datos, el cliente instanció la librería oficial PocketBase configurando una URL dinámica (pbUrl). Esta lógica evalúa si existe una variable de entorno PUBLIC_POCKETBASE_URL o, en su defecto, detecta dinámicamente en el navegador el protocolo y la dirección IP desde donde se cargó la aplicación, apuntando directamente al puerto 8095.



Reactividad y Estado Global con Runes de Svelte 5

Declaración de Estado Reactivo y Propiedades Derivadas

App.svelte adopta el nuevo paradigma de reactividad de **Svelte 5** mediante el uso de *Runes*:

- **Estado Mutable ($state):** Se declaran variables de estado reactivas como estaAutenticado (control de sesión), activos (lista del inventario oficial), shadowItList (equipos en cuarentena), search (búsqueda global), filtroTipo (filtro por tipo de equipo) y sucursal (filtro por sede).

- **Propiedades Derivadas ($derived y $derived.by):** En lugar de recalcular manualmente los valores en cada renderizado, las propiedades derivadas recalculan de forma automática la información relevante cuando su estado dependiente cambia:
    - shadowFormateados: Transforma los registros crudos de la tabla shadow_it para que coincidan con la estructura unificada de los activos del sistema.
    - filtered: Aplica en tiempo real los filtros de texto y categoría (PC, Servidor, NVR, Shadow IT o Crítico < 50).
    - stats: Calcula automáticamente el score promedio general de la empresa y la cantidad total de equipos críticos, PCs, NVRs y servidores.



```
let estaAutenticado = $state(false);

let activos = $state<any[]>([]);
let shadowItList = $state<any[]>([]);
let cargando = $state(false);
let errorRed = $state<string | null>(null);

let search = $state("");
let filtroTipo = $state("Todos");
let filtroNivel = $state("Todos");
let sucursales = $state<string[]>(
  typeof window !== 'undefined'
    ? JSON.parse(localStorage.getItem('app_sucursales') || '["Sede Central", "Sucursal Norte", "Sucursal Sur", "Bodega Central"]')
    : ["Sede Central", "Sucursal Norte", "Sucursal Sur", "Bodega Central"]
);
let sucursal = $derived(sucursales[0] || "Sede Central");

let perPage = $state(6);
let currentPage = $state(1);

let modalActivo = $state<any>(null);
let showModal = $state(false);

let modalEditarActivo = $state(null);
let showEditModal = $state(false);

let esAdmin = $derived(pb.authStore.model?.role === 'admin');
let urlVerId = $state<string | null>(null);
```



```
let shadowFormateados = $derived(shadowItList.map((item: any) => ({
  id: item.id,
  hostname: `SHADOW-${item.mac_detectada?.replaceAll(':', '') || 'DEV'}`,
  ip: item.ip_detectada,
  mac: item.mac_detectada,
  tipo: 'Shadow IT',
  estado_ciclo: item.estado || 'Detectado',
  isShadow: true,
  score_seguridad: 0,
  ubicacion: 'Red Wi-Fi (No autorizada)',
  responsable: 'Desconocido',
  tipo_sugerido: item.tipo_sugerido,
  fabricante: item.fabricante,
  puertos_abiertos: item.puertos_abiertos,
  vulnerabilidades: item.vulnerabilidades
})));

let activosValidos = $derived(activos.filter(a => a.ip !== "192.168.1.1"));
let todosLosDispositivos = $derived([...activosValidos, ...shadowFormateados]);
```

```
let filtered = $derived.by(() => {
  const q = search.toLowerCase();

  if (filtroTipo === "Shadow IT") {
    return shadowFormateados.filter(a =>
      !q ||
      (a.hostname || "").toLowerCase().includes(q) ||
      (a.ip || "").includes(q) ||
      (a.mac || "").toLowerCase().includes(q) ||
      (a.responsable || "").toLowerCase().includes(q)
    );
  }

  return activosValidos.filter(a => {
    const matchSearch = !q ||
      (a.hostname || "").toLowerCase().includes(q) ||
      (a.ip || "").includes(q) ||
      (a.mac || "").toLowerCase().includes(q) ||
      (a.responsable || "").toLowerCase().includes(q);

    let matchTipo = true;
    if (filtroTipo === "Todos") {
      matchTipo = true;
    } else if (filtroTipo === "Crítico") {
      matchTipo = a.tipo !== 'Otros' && (a.score_seguridad ?? a.score) < 50;
    } else {
      matchTipo = a.tipo === filtroTipo;
    }

    return matchSearch && matchTipo;
  });
});
```

```
let stats = $derived.by(() => {
  const oficiales = activosValidos;
  const conScore = oficiales.filter(a => a.tipo !== 'Otros' && (a.score_seguridad ?? a.score) != null);
  const totalScore = conScore.reduce((sum, a) => sum + (a.score_seguridad ?? a.score), 0);
  const promedio = conScore.length > 0 ? Math.round(totalScore / conScore.length) : 0;
  return {
    total: oficiales.length,
    promedio: promedio,
    criticos: conScore.filter(a => (a.score_seguridad ?? a.score) < 50).length,
    shadow: shadowFormateados.length,
    nvrs: oficiales.filter(a => a.tipo === "NVR").length,
    pcs: oficiales.filter(a => a.tipo === "PC").length,
    servidores: oficiales.filter(a => a.tipo === "Servidor").length,
    otros: oficiales.filter(a => a.tipo === "Otros").length,
  };
});
```

Operaciones con la API y Aprobación de Shadow IT

Métodos de Sincronización y Promoción de Activos

En este bloque se definen las funciones asíncronas de interacción directa con la base de datos:

- **actualizarDesdePocketBase(silencioso):** Consulta en paralelo (Promise.all) las colecciones activos y shadow_it. Aplica filtros dinámicos si se ha seleccionado una sede específica y despliega alertas visuales (*Toasts*) sobre el estado de la sincronización.
- **aprobarShadowIt(item, tipoManual):** Función crítica restringida exclusivamente a usuarios con rol admin. Permite tomar un dispositivo detectado en cuarentena (*Shadow IT*), asignarle un número de serie único derivado de su dirección MAC, crear su registro oficial en la colección activos, crear su instancia inicial en evaluaciones_red y eliminarlo de la cola de dispositivos no autorizados.




```
async function actualizarDesdePocketBase(silencioso = false) {
  cargando = true;
  errorRed = null;
  try {
    const opcionesActivos: any = {
      sort: '-created',
      requestKey: null
    };

    if (sucursal && sucursal !== 'Todas') {
      opcionesActivos.filter = `ubicacion = "${sucursal}"`;
    }

    const [recordsActivos, recordsShadow] = await Promise.all([
      pb.collection('activos').getFullList(opcionesActivos),
      pb.collection('shadow_it').getFullList({
        sort: '-created',
        requestKey: null
      })
    ]);

    activos = recordsActivos;
    shadowItList = recordsShadow;

    if (!silencioso) {
      triggerToast(`Sincronización completa: ${activosValidos.length} elementos cargados`, "is-success");
    }
  } catch (err: any) {
    if (err?.name === 'AbortError' || err?.status === 0) return;

    console.error("--- ERROR REAL DE POCKETBASE ---", err);
    const mensajeReal = err?.message || JSON.stringify(err);
    errorRed = "Error de BD: " + mensajeReal;
    if (!silencioso) triggerToast("Error: " + mensajeReal, "is-danger");
  } finally {
```

```
async function aprobarShadowIt(item: any, tipoManual: string = 'PC') {
  const usuarioActual = pb.authStore.model;
  if (!usuarioActual || usuarioActual.role !== 'admin') {
    triggerToast("Acceso denegado: Se requiere rol de administrador para aprobar dispositivos", "is-danger");
    return;
  }

  try {
    if (!item) {
      triggerToast("Error: El dispositivo seleccionado es inválido", "is-danger");
      return;
    }

    const macReal = item.mac || '00:00:00:00:00:00';
    const ipReal = item.ip || '0.0.0.0';
    const serialUnico = `SN-${macReal.replaceAll(':', '').toUpperCase()}`;
    const hostnameAsignado = item.hostname && !item.hostname.startsWith('SHADOW-')
      ? item.hostname
      : `PC-${ipReal.replaceAll('.', '-')}`;

    const nuevoActivo = {
      hostname: hostnameAsignado,
      ip: ipReal,
      mac: macReal,
      tipo: tipoManual,
      estado_ciclo: 'Operativo',
      score_seguridad: 100,
      ubicacion: sucursal,
      responsable: 'Asignado Manualmente',
      n_serie: serialUnico
    };

    const resActivo = await pb.collection('activos').create(nuevoActivo);
```

Ciclo de Vida, Tiempo Real y deep linking (QR)

Gestión de Eventos en Tiempo Real (Realtime SSE) y Verificación QR

El bloque del ciclo de vida (onMount y onDestroy) ejecuta tres tareas fundamentales:

1. **Verificación de Enlace QR (Deep Linking):** Lee la URL del navegador buscando el parámetro ?ver=ID. Si el usuario escanea un código QR sin haber iniciado sesión previa, guarda el parámetro en sessionStorage para abrir automáticamente la ficha técnica del activo una vez completado el login.
2. **Suscripción en Tiempo Real (SSE):** Se conecta a los canales en tiempo real de PocketBase (pb.collection('activos').subscribe('*') y pb.collection('shadow_it').subscribe('*')). Esto permite que cuando el agente Python o los workers Node.js modifiquen un activo o detecten un nuevo dispositivo en red, la pantalla del administrador se actualice inmediatamente sin requerir refrescar la página.
3. **Limpieza de Recursos (onDestroy):** Cancela todas las suscripciones activas al desmontar el componente para evitar fugas de memoria (*memory leaks*).

```
onMount(async () => {
  estaAutenticado = pb.authStore.isValid;

  const urlParams = new URLSearchParams(window.location.search);
  const verId = urlParams.get('ver');

  if (verId) {
    urlVerId = verId;
  }

  if (verId && !estaAutenticado) {
    sessionStorage.setItem('pending_ver', verId);
  }

  if (estaAutenticado) {
    await actualizarDesdePocketBase(!!verId);

    if (verId) {
      await abrirAssetPorId(verId);
    }
  }

  try {
    await pb.collection('activos').subscribe('*', (e) => {
      if (e.action === 'create' && !activos.some(a => a.id === e.record.id)) activos = [e.record, ...activos];
      else if (e.action === 'update') activos = activos.map(a => a.id === e.record.id ? e.record : a);
      else if (e.action === 'delete') activos = activos.filter(a => a.id !== e.record.id);
    });
    await pb.collection('shadow_it').subscribe('*', (e) => {
      if (e.action === 'create' && !shadowItList.some(s => s.id === e.record.id)) shadowItList = [e.record, ...shadowItList];
      else if (e.action === 'update') shadowItList = shadowItList.map(s => s.id === e.record.id ? e.record : s);
      else if (e.action === 'delete') shadowItList = shadowItList.filter(s => s.id !== e.record.id);
    });
  } catch (err) {
    console.error("Error suscribiendo realtime PocketBase:", err);
```

```
  onDestroy(() => {
    pb.collection('activos').unsubscribe('*').catch(() => {});
    pb.collection('shadow_it').unsubscribe('*').catch(() => {});
  });
</script>
```

Renderizado de la Interfaz y Enrutamiento de Modales

Plantilla de Renderizado HTML y Ruteo Dinámico de Modales

La plantilla visual evalúa condicionalmente el estado de autenticación:

- Si estaAutenticado es falso, muestra únicamente el componente <LoginView/>.
- Si el usuario inició sesión, monta la barra de navegación <Navbar/>, el resumen de indicadores <StatsLevel/>, el menú lateral <SidebarMenu/> y conmuta entre el vista ejecutiva <DashboardView/> y la tabla <InventoryTable/>.
- **Modales Dinámicos:** Según la categoría del activo seleccionado (modalActivo), despliega el modal especializado:
    - ServerNvrModal: Si el tipo es *Servidor* o *NVR*.
    - OtherAssetModal: Si el tipo pertenece a la categoría neutra *Otros*.
    - DetailModal: Para estaciones de trabajo *PC*.
    - EditAssetModal: Si un administrador solicita editar los campos de la ficha técnica.

```
{#if !estaAutenticado}
  <LoginView onLoginExito={async () => {
    estaAutenticado = true;

    const pendingVer = sessionStorage.getItem('pending_ver');
    if (pendingVer) {
      urlVerId = pendingVer;
      sessionStorage.removeItem('pending_ver');
      window.history.replaceState({}, document.title, window.location.pathname);
      await actualizarDesdePocketBase(true);
      await abrirAssetPorId(pendingVer);
    } else {
      await actualizarDesdePocketBase(false);
      triggerToast("Sesión iniciada correctamente", "is-success");
    }
  }} />
{:else}
  {#if !urlVerId}
    <Navbar bind:sucursal {sucursales} {activos} {stats} {esAdmin} onActualizarSedes={actualizarSedes} onSincronizar={() => actua

    <main class="container mt-5 px-4 pb-6">
      <StatsLevel {stats} {scoreClass} />

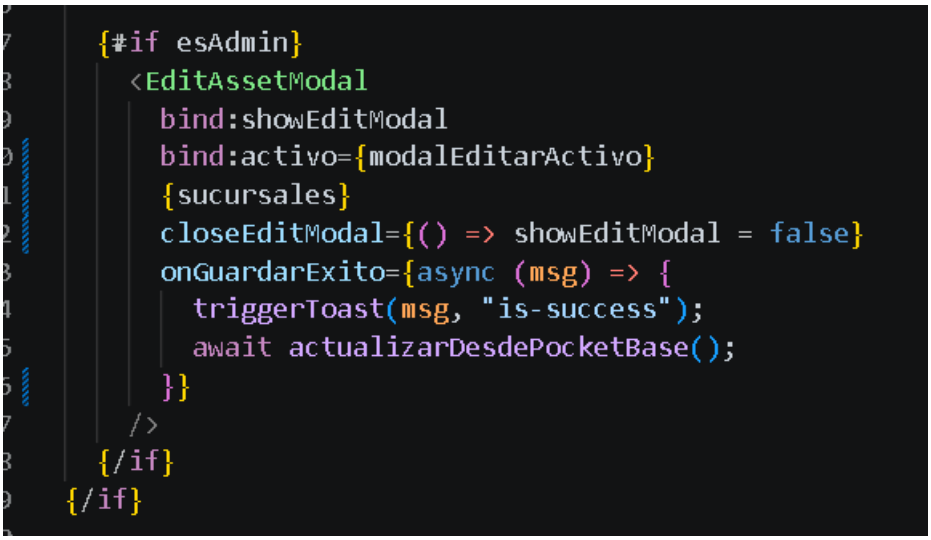
      <div class="columns">
        <div class="column is-3">
          <SidebarMenu bind:vista bind:search bind:filtroTipo {activosValidos} {shadowFormateados} {onSearchInput} {setTipo} />
        </div>

        <div class="column is-9">
          {#if errorRed}
            <div class="notification is-danger is-light">
              <button class="delete" aria-label="Cerrar" onclick={() => errorRed = null}></button>
              {errorRed}
            </div>
          {/if}
```

```
          {#if vista==="Dashboard"}
            <DashboardView {activosValidos} {shadowFormateados} {stats} {openModal} />
          {:else if vista==="Inventario"}
            <InventoryTable {paginated} {totalPages} {currentPage} {goPage} {scoreClass} {rowClass} {openModal} {pages} />
          {/if}
        </div>
      </div>
    </main>
  {:else}
    <div style="min-height: 100vh; background-color: #0f172a;"></div>
  {/if}

```

```
{#if modalActivo && showModal}
  {#if modalActivo.tipo === 'Servidor' || modalActivo.tipo === 'NVR'}
    <ServerNvrModal
      bind:showModal
      {modalActivo}
      {closeModal}
      onSolicitarRescaneo={(activo) => triggerToast(`Re-escaneo solicitado para ${activo.hostname}`, "is-info")}
      onAbrirEditar={esAdmin ? openEditModal : undefined}
    />
  {:else if modalActivo.tipo === 'Otros'}
    <OtherAssetModal
      bind:showModal
      {modalActivo}
      {closeModal}
      onAbrirEditar={esAdmin ? openEditModal : undefined}
    />
  {:else}
    <DetailModal
      bind:showModal
      {modalActivo}
      {closeModal}
      onAprobar={esAdmin ? aprobarShadowIt : undefined}
      onAbrirEditar={esAdmin ? openEditModal : undefined}
    />
  {/if}
{/if}
```



```
    {#if esAdmin}
      <EditAssetModal
        bind:showEditModal
        bind:activo={modalEditarActivo}
        {sucursales}
        closeEditModal={() => showEditModal = false}
        onGuardarExito={async (msg) => {
          triggerToast(msg, "is-success");
          await actualizarDesdePocketBase();
        }}
      />
    {/if}
  {/if}
```

Pantalla de Autenticación (LoginView.svelte)

Gestión de Credenciales y Autenticación de Usuarios

El componente LoginView.svelte proporciona la interfaz visual y lógica de inicio de sesión para restringir el acceso al sistema. Utiliza el SDK de PocketBase para autenticar al usuario contra la colección users mediante el método pb.collection('users').authWithPassword(email, password).

Su implementación incluye:

- **Estados Reactivos ($state):** Gestionan de forma reactiva el correo ingresado (email), la contraseña (password), los mensajes de error en caso de credenciales inválidas (errorMsg) y un indicador de carga (cargando) para deshabilitar el botón y mostrar una animación durante la petición HTTP.
- **Callback de Éxito:** Si la autenticación es correcta, ejecuta la función onLoginExito() pasada por propiedades, o recarga la página redirigiendo al dashboard principal.

```svelte
<script lang="ts">
  import PocketBase from 'pocketbase';

  let { onLoginExito }: { onLoginExito?: () => void } = $props();

  const pbUrl = import.meta.env.PUBLIC_POCKETBASE_URL ||
    (typeof window !== 'undefined' ? `${window.location.protocol}//${window.location.hostname}:8095` : '');
  const pb = new PocketBase(pbUrl);

  let email = $state('');
  let password = $state('');
  let errorMsg = $state('');
  let cargando = $state(false);

  async function handleLogin(e: Event) {
    e.preventDefault();
    cargando = true;
    errorMsg = '';

    try {
      await pb.collection('users').authWithPassword(email, password);
      if (onLoginExito) {
        onLoginExito();
      } else {
        window.location.href = '/';
      }
    } catch (err) {
      console.error(err);
      errorMsg = 'Credenciales incorrectas o usuario no autorizado.';
    } finally {
      cargando = false;
    }
  }
</script>
```

Barra de Navegación Superior (Navbar.svelte)

Selector de Sucursales, Sincronización y Control de Sesión

Navbar.svelte es la barra de navegación superior fija del sistema. Sus funciones principales son:

- **Selector de Sucursales:** Permite cambiar de manera reactiva la sede activa mediante un menú desplegable (sucursal), actualizando el filtro general de activos.
- **Sincronización Manual:** Botón con icono de recarga (onSincronizar) para forzar la actualización de los datos desde PocketBase.
- **Gestión de Sedes:** Exclusivo para administradores (esAdmin). Abre el modal de configuración de sucursales (SedeModal).
- **Menú de Usuario y Cierre de Sesión:** Muestra el nombre o correo del usuario autenticado (displayName) obtenido desde pb.authStore.model, permitiendo limpiar el almacén de sesión (pb.authStore.clear()) y recargar la página para salir de forma segura.

```svelte
<script lang="ts">
  import PocketBase from 'pocketbase';
  import SedeModal from './SedeModal.svelte';

  let {
    sucursal = $bindable(),
    sucursales = [],
    activos = [],
    esAdmin = false,
    onActualizarSedes,
    stats,
    onSincronizar
  } = $props();

  let showSedeModal = $state(false);

  const pbUrl = import.meta.env.PUBLIC_POCKETBASE_URL ||
    (typeof window !== 'undefined' ? `${window.location.protocol}//${window.location.hostname}:8095` : '');
  const pb = new PocketBase(pbUrl);

  let displayName = $derived(pb.authStore.model?.name || pb.authStore.model?.email || 'Admin TI');
</script>
```

Panel de Indicadores Globales (StatsLevel.svelte)

Resumen Ejecutivo de Métricas Corporativas (KPIs)

StatsLevel.svelte renderiza cuatro tarjetas de resumen ejecutivo ubicadas en la parte superior del dashboard:

1. **Total Activos:** Muestra el volumen general y su desglose detallado por tipo (PCs, Servidores, NVRs y Otros).
2. **Score Promedio:** Calcula y muestra la puntuación global de ciberseguridad, aplicando colores dinámicos (verde, amarillo o rojo) y una barra de progreso porcentual mediante la función auxiliar scoreClass.
3. **Equipos en Riesgo:** Alerta numérica y etiqueta visual para los dispositivos con un score crítico menor a 50 puntos.
4. **Shadow IT / Cuarentena:** Conteo de dispositivos no autorizados detectados en la red mediante análisis OUI. Cambia de color e intensidad si existen elementos pendientes de revisión.

```svelte
Runes mode
<script lang="ts">
  let { stats, scoreClass } = $props();
</script>

<nav class="level mb-5">
  <div class="level-item has-text-centered">
    <div class="box has-background-white-bis w-100">
      <p class="heading">Total Activos</p>
      <p class="title is-4 has-text-grey-darker">{stats.total}</p>
      <p class="is-size-7 has-text-grey">
        <i class="fas fa-laptop mr-1"></i> {stats.pcs} PCs • {stats.servidores} Servers • {stats.nvrs} NVRs • {stats.otros ?? 0} Otros
      </p>
    </div>
  </div>
  <div class="level-item has-text-centered">
    <div class="box has-background-white-bis w-100">
      <p class="heading">Score Promedio</p>
      <p class="title is-4 {stats.promedio >= 80 ? 'has-text-success' : stats.promedio >= 50 ? 'has-text-warning' : 'has-text-danger'}">
        {stats.promedio}/100
      </p>
      <progress class="progress is-small {scoreClass(stats.promedio)}" value={stats.promedio} max="100"></progress>
    </div>
  </div>
  <div class="level-item has-text-centered">
    <div class="box has-background-white-bis w-100">
      <p class="heading">Equipos en Riesgo</p>
      <p class="title is-4 has-text-danger">{stats.criticos}</p>
      <span class="tag is-danger is-light">
        <i class="fas fa-exclamation-triangle mr-1"></i> Score &lt; 50
      </span>
    </div>
  </div>
  <div class="level-item has-text-centered">
    <div class="box {stats.shadow > 0 ? 'has-background-grey-lighter' : 'has-background-white-bis'} w-100">
      <p class="heading">Shadow IT / Cuarentena</p>
      <p class="title is-4 {stats.shadow > 0 ? 'has-text-grey-dark' : 'has-text-info'}">{stats.shadow}</p>
```

Menú Lateral y Filtros de Seguridad (SidebarMenu.svelte)

Navegación de Vistas y Filtrado Granular

SidebarMenu.svelte cumple un rol dual en la interfaz:

1. **Navegación Principal:** Permite alternar la vista activa entre el **Dashboard** ejecutivo y la tabla completa de **Inventario**.
2. **Filtros Granulares de Seguridad:** Contiene una barra de búsqueda rápida por IP u Hostname y botones de selección rápida que calculan dinámicamente el conteo de elementos para cada categoría (Todos, PCs, Servidores, NVRs, Otros, Shadow IT y Equipos Críticos).

```svelte
<script lang="ts">
  let { vista = $bindable(), search = $bindable(), filtroTipo = $bindable(), activosValidos, shadowFormateados = [], onSearchInpu

  let criticosCount = $derived(activosValidos.filter((a: any) => !a.isShadow && a.tipo !== 'Otros' && (a.score_seguridad ?? a.sco
  let pcsCount = $derived(activosValidos.filter((a: any) => !a.isShadow && a.tipo === 'PC').length);
  let serversCount = $derived(activosValidos.filter((a: any) => !a.isShadow && a.tipo === 'Servidor').length);
  let nvrsCount = $derived(activosValidos.filter((a: any) => !a.isShadow && a.tipo === 'NVR').length);
  let otrosCount = $derived(activosValidos.filter((a: any) => !a.isShadow && a.tipo === 'Otros').length);
  let shadowCount = $derived(shadowFormateados.length);
</script>
```

Vista de Dashboard Ejecutivo (DashboardView.svelte)

Tarjetas Resumen de Alertas y Acceso Rápido a Revisión

DashboardView.svelte organiza la vista ejecutiva en tres tarjetas temáticas con codificación de colores:

- **Tarjeta Roja (Críticos):** Filtra y muestra los 3 dispositivos oficiales con score menor a 50 puntos, permitiendo abrir directamente su modal de revisión.
- **Tarjeta Amarilla (Medios):** Muestra los equipos en riesgo moderado (Score entre 50 y 79).
- **Tarjeta Gris (Shadow IT):** Lista los dispositivos no autorizados detectados en cuarentena para su respectiva auditoría y aprobación.



```
<script lang="ts">
  let { activosValidos, shadowFormateados, stats, openModal } = $props();
</script>

<div class="columns is-multiline mb-2">
  <!-- Tarjeta Roja: Críticos -->
  <div class="column is-4">
    <article class="message is-danger is-small mb-3">
      <div class="message-header">
        <p><i class="fas fa-exclamation-triangle mr-1"></i> Crítico (&lt;50)</p>
        <span class="tag is-danger is-light">{stats.criticos}</span>
      </div>
      <div class="message-body p-3" style="min-height: 140px;">
        {#each activosValidos.filter((a: any)=>!a.isShadow && a.tipo !== 'Otros' && (a.score_seguridad ?? a.score)!=null && (a.sc
          <div class="is-flex is-justify-content-space-between is-align-items-center mb-2">
            <div>
              <strong>{a.hostname}</strong><br>
              <small class="has-text-grey">({a.ip})</small>
            </div>
            <button class="button is-small is-danger is-light" onclick={()=>openModal(a)}>Revisar</button>
          </div>
        {/each}
        {#if stats.criticos===0}<p class="has-text-grey">Sin equipos críticos.</p>{/if}
      </div>
    </article>
  </div>
```

Tabla de Inventario Paginada (InventoryTable.svelte)

Listado General, Puntos de Estado y Paginación

InventoryTable.svelte renderiza los registros en formato tabular avanzado:

- **Puntos de Estado Animados (estadoDotClass):** Muestra un indicador visual dinámico según el ciclo de vida del activo (*Operativo*, *Mantenimiento*, *Cuarentena* o *Baja*).
- **Etiquetas de Tipo y Score:** Aplica estilos condicionales diferenciados para PCs, Servidores, NVRs y dispositivos Shadow IT, formateando el score de seguridad o mostrando un guion neutro para la categoría *Otros*.

- **Paginación Dinámica:** Controla la navegación por bloques de registros mediante los botones de página anterior/siguiente.

```svelte
<script lang="ts">
  let { paginated, totalPages, currentPage, goPage, scoreClass, rowClass, openModal, pages } = $props();

  function estadoDotClass(estado: string) {
    switch (estado) {
      case 'Operativo': return 'dot-success';
      case 'Mantenimiento': return 'dot-warning';
      case 'Cuarentena': return 'dot-danger';
      case 'Baja': return 'dot-dark';
      default: return 'dot-success';
    }
  }
</script>
```

Vista de Generación de Actas PDF y Código QR (ActaView.svelte)

Generación de Actas de Entrega/Devolución y Códigos QR

ActaView.svelte permite generar actas oficiales de entrega o devolución de equipos optimizadas para formato físico A4 horizontal. Sus características principales son:

- **Carga de Relaciones:** Al montar el componente (onMount), consulta en PocketBase el activo y sus evaluaciones asociadas de red y endpoints.
- **Adaptabilidad Criptográfica:** Detecta automáticamente si el sistema operativo es Linux (asignando cifrado **LUKS**) o Windows (asignando **BitLocker**).
- **Generación de Código QR Integrado:** Renderiza un código QR mediante una API externa utilizando el identificador único del activo para garantizar la trazabilidad física.
- **Automatización de Impresión:** Dispara de forma automática la función nativa de impresión del navegador (window.print()) tras un breve retardo de carga.

```svelte
  onMount(async () => {
    try {
      activo = await pb.collection('activos').getOne(id);

      try {
        evalEndpoint = await pb.collection('evaluaciones_endpoints').getFirstListItem(`relation="${id}"`, { sort: '-created' });
      } catch (e) { evalEndpoint = null; }

      try {
        auditoriaRed = await pb.collection('evaluaciones_red').getFirstListItem(`relation="${id}"`, { sort: '-created' });
      } catch (e) { auditoriaRed = null; }

      cargando = false;

      setTimeout(() => {
        window.print();
      }, 600);
    } catch (err) {
      console.error('Error al cargar datos del acta:', err);
      cargando = false;
    }
  });
```

Modal de Ficha y Re-escaneo de PCs / Endpoints (DetailModal.svelte)

Auditoría Local, Cifrado, EDR y Re-escaneo en Tiempo Real

DetailModal.svelte gestiona la visualización detallada de las estaciones de trabajo (PCs):

- **Carga de Evaluaciones Cruzadas:** Al abrirse el modal, consulta en PocketBase las colecciones de evaluaciones_red y evaluaciones_endpoints vinculadas al activo mediante su ID, IP o Hostname.
- **Suscripción Reactiva en Tiempo Real (SSE):** Se suscribe a los cambios del activo y sus evaluaciones para reflejar de forma instantánea los resultados de auditoría en pantalla.
- **Re-escaneo Forzado:** Permite al administrador activar la bandera forzar_escaneo_manual: true, enviando una instrucción al agente Python del endpoint para actualizar su postura de seguridad al instante.
- **Generación de Etiquetas QR Individuales:** Cuenta con una función para abrir una ventana optimizada para impresoras térmicas de etiquetas físicas con el código QR del activo.

```
35    async function solicitarReescaneo() {
36      if (!activoLocal?.id) return;
37      escaneando = true;
38      tiempoInicioEscaneo = Date.now();
39
40      if (timeoutRespaldo) clearTimeout(timeoutRespaldo);
41      timeoutRespaldo = setTimeout(() => {
42        escaneando = false;
43      }, 25000);
44
45      try {
46        await pb.collection('activos').update(activoLocal.id, {
47          forzar_escaneo_manual: true
48        });
49        if (onSolicitarRescaneo) {
50          onSolicitarRescaneo(activoLocal);
51        }
52      } catch (err) {
53        console.error('Error al solicitar re-escaneo:', err);
54        escaneando = false;
55        if (timeoutRespaldo) clearTimeout(timeoutRespaldo);
56      }
57    }
58
59    $effect(() => {
60      if (!modalActivo?.id) {
61        activoLocal = null;
62        auditoriaRed = null;
63        evalEndpoint = null;
64        escaneando = false;
65        if (timeoutRespaldo) clearTimeout(timeoutRespaldo);
66        return;
67      }
68
69      activoLocal = modalActivo;
```

Modal de Servidores y NVRs (ServerNvrModal.svelte)

Auditoría de Red, Puertos Abiertos y Contraseñas por Defecto

ServerNvrModal.svelte está diseñado específicamente para la inspección de infraestructura crítica (Servidores y Grabadores de Video NVR):

- **Análisis de Puertos Críticos:** Interpreta y clasifica los puertos de red abiertos (22 para SSH, 23 para Telnet, 80/443 para HTTP/HTTPS, 554 para RTSP de video, y 3389 para RDP), aplicando etiquetas de advertencia según el nivel de riesgo.
- **Detección de Credenciales por Defecto:** Evalúa si el dispositivo mantiene contraseñas de fábrica activas en sus interfaces web o servicios expuestos.
- **Re-auditoría de Red:** Permite forzar un nuevo escaneo de puertos y servicios mediante el worker de red Node.js.

```
function infoPuerto(puerto: string) {
  const p = String(puerto).trim();
  switch (p) {
    case '22': return { nombre: 'SSH (22)', tagClass: 'is-info' };
    case '23': return { nombre: 'Telnet (23)', tagClass: 'is-danger' };
    case '80': return { nombre: 'HTTP (80)', tagClass: 'is-warning' };
    case '443': return { nombre: 'HTTPS (443)', tagClass: 'is-success' };
    case '554': return { nombre: 'RTSP Video (554)', tagClass: 'is-warning' };
    case '3389': case '13389': return { nombre: `RDP (${p})`, tagClass: 'is-danger' };
    default: return { nombre: `Puerto ${p}`, tagClass: 'is-dark' };
  }
}
```

Modal de Edición de Activos (EditAssetModal.svelte)

Modificación de Metadatos y Asignación de Sedes

EditAssetModal.svelte permite a los administradores actualizar la información de los activos:

- **Campos Editables:** Hostname, número de serie (N° Serie / RF-01), tipo de dispositivo, estado en el ciclo de vida (*Operativo*, *Mantenimiento*, *Baja*, *Cuarentena*), ubicación (sede) y responsable asignado.
- **Protección de Datos Clave:** Las direcciones IP y MAC se muestran bloqueadas (disabled readonly) para evitar inconsistencias con los agentes de red.
- **Lógica de Categoría Neutra:** Si un activo es modificado a la categoría "Otros", el sistema asigna automáticamente un score de seguridad nulo para excluirlo del cálculo del promedio corporativo global.

```
45    async function guardarCambios() {
46      if (!activo?.id) return;
47      guardando = true;
48      try {
49        const payload: any = {
50          hostname,
51          n_serie: nSerie,
52          tipo,
53          estado_ciclo: estadoCiclo,
54          ubicacion,
55          responsable
56        };
57
58        // Si se cambia a la categoría "Otros", se asigna score nulo para excluirlo del promedio global
59        if (tipo === 'Otros') {
60          payload.score_seguridad = null;
61        }
62
63        await pb.collection('activos').update(activo.id, payload);
64
65        if (onGuardarExito) {
66          onGuardarExito(`Activo ${hostname} actualizado correctamente`);
67        }
68        closeEditModal();
69      } catch (err: any) {
70        console.error("Error al actualizar activo:", err);
71        alert("Error al guardar cambios: " + (err?.message || "Error de conexión"));
72      } finally {
73        guardando = false;
74      }
75    }
```

Modal de Gestión de Sedes (SedeModal.svelte)

Creación, Edición y Validación de Registros Huérfanos

SedeModal.svelte es un componente exclusivo para administradores que gestiona las sucursales de la empresa:

- **Adición y Eliminación:** Permite registrar nuevas sedes o eliminar existentes.
- **Prevención de Registros Huérfanos:** Al intentar eliminar una sede, verifica si existen dispositivos asignados a ella y emite una advertencia al operador para evitar que los activos queden sin sucursal asignada en los filtros del dashboard.

```
function agregarSede() {
  const trimmed = nuevaSede.trim();
  if (trimmed && !listaLocal.includes(trimmed)) {
    listaLocal = [...listaLocal, trimmed];
    nuevaSede = "";
  }
}
```

```
function eliminarSede(index: number) {
  const sedeAEliminar = listaLocal[index];

  // Validar si hay activos usando esta sede
  const activosEnSede = activos.filter(a => a.ubicacion === sedeAEliminar);

  if (listaLocal.length <= 1) {
    alert("Debe existir al menos una sede en el sistema.");
    return;
  }

  if (activosEnSede.length > 0) {
    const confirmar = confirm(
      `⚠️ ADVERTENCIA: Hay ${activosEnSede.length} activo(s) registrado(s) en "${sedeAEliminar}".\n\nSi eliminas la sede, esos
    );
    if (!confirmar) return;
  }

  listaLocal = listaLocal.filter((_, i) => i !== index);
}
```

Utilidad de Generación de Actas PDF y QR Local (pdf-helper.ts)

Construcción Dinámica de Actas y Renderizado de Códigos QR

El archivo pdf-helper.ts centraliza la lógica para generar actas oficiales de entrega o devolución de activos optimizadas para formato físico:

- **Generación de QR Local:** Utiliza la librería qrcode-generator para codificar la URL directa de verificación del activo (/?ver=ID).
- **Estructura Condicional de Postura:** Evalúa si el activo es un computador (mostrando estado de **BitLocker/LUKS**, **EDR**, parches y administradores locales) o infraestructura crítica como Servidores y NVRs (mostrando contraseñas por defecto, puertos abiertos y firmware).
- **Automatización de Impresión:** Inserta el código HTML estructurado en una nueva ventana del navegador y ejecuta de manera automática la instrucción window.print().

```
export function generarActaPdf(
    activo: any,
    tipoActa: 'entrega' | 'devolucion' = 'entrega',
    evaluacionEndpoint?: any | null,
    evaluacionRed?: any | null
) {
    if (!activo) return;

    const fecha = new Date().toLocaleDateString('es-CL', {
        day: '2-digit',
        month: 'long',
        year: 'numeric'
    });

    const titulo = tipoActa === 'entrega' ? 'Acta de Entrega' : 'Acta de Devolución';
    const qrUrl = generarQrLocal(activo);
    const score = Number(activo.score_seguridad ?? activo.score ?? 100);
```

Ruta de Inicio de Sesión en SvelteKit (routes/login/+page.svelte)

Vista Dedicada de Login en SvelteKit

Esta página gestiona el inicio de sesión directo en la ruta /login:

- Se conecta a la colección users de PocketBase mediante authWithPassword.
- Maneja estados reactivos de carga y despliega notificaciones visuales en caso de credenciales incorrectas.

```
async function handleLogin(e: Event) {
  e.preventDefault();
  cargando = true;
  errorMsg = '';

  try {
    await pb.collection('users').authWithPassword(email, password);
    window.location.href = '/';
  } catch (err) {
    console.error(err);
    errorMsg = 'Credenciales incorrectas o usuario no autorizado.';
  } finally {
    cargando = false;
  }
}
```

Ruta Pública de Verificación por Código QR (routes/qr/[id]/+page.svelte)

Auditoría en Terreno sin Autenticación Previa

Permite a cualquier operador auditar un activo escaneando su etiqueta física con un teléfono móvil:

- Lee el parámetro de ruta id y consulta en tiempo real las colecciones de activos, evaluaciones_red y evaluaciones_endpoints.
- Despliega la ficha técnica, el score de seguridad y ofrece un botón directo para generar el acta PDF inmutable en terreno.



```
$effect(() => {
  if (activoId) {
    cargando = true;
    errorMsg = '';

    pb.collection('activos')
      .getOne(activoId)
      .then((resActivo) => {
        activo = resActivo;

        const filtro = `relation="${activoId}" || host_asociado="${activo.hostname}" || ip_asociada="${activo.ip}"`;

        pb.collection('evaluaciones_red')
          .getFirstListItem(filtro, { sort: '-created' })
          .then((resRed) => (auditoriaRed = resRed))
          .catch(() => (auditoriaRed = null));

        pb.collection('evaluaciones_endpoints')
          .getFirstListItem(`relation="${activoId}"`, { sort: '-created' })
          .then((resEnd) => (evalEndpoint = resEnd))
          .catch(() => (evalEndpoint = null));
      })
      .catch((err) => {
        console.error(err);
        errorMsg = 'No se encontró el activo en la base de datos o el enlace es inválido.';
      })
      .finally(() => {
        cargando = false;
      });
  }
});
```

Módulo Backend & Workers (Node.js)

Worker de Descubrimiento de Red y Shadow IT (worker-network.js)

Escaneo ARP, Análisis OUI y Sondeo de Puertos

worker-network.js es el servicio encargado de vigilar la red corporativa de forma autónoma:

- **Escaneo Nmap y Tabla ARP:** Ejecuta sondeos en la subred y lee la tabla del kernel (/proc/net/arp) para extraer las direcciones IP y MAC activas.
- **Clasificación por OUI:** Compara los primeros octetos de la MAC contra una base de datos interna para identificar si el equipo es un NVR, impresora, servidor o PC. Si el OUI es desconocido, realiza sondeos TCP a puertos clave (554, 8000, 445, 631, 22) o banner-grabbing HTTP web.
- **Cuarentena Shadow IT:** Si la MAC detectada no se encuentra en la colección oficial de activos, crea un registro automático en la tabla shadow_it para su revisión administrativa.

```
function escanearRedNmap() {
  return new Promise((resolve) => {
    const dispositivos = [];
    const targetSubnet = process.env.TARGET_SUBNET || '192.168.10.0/24';
    const subnetBase = targetSubnet.substring(0, targetSubnet.lastIndexOf('.'));

    console.log(`[DEBUG] Ejecutando sondeo unprivileged en la subred: ${targetSubnet}`);

    // nmap unprivileged con puertos clave para forzar al kernel a poblar /proc/net/arp con todos los hosts
    exec(`nmap -p 80,445,22 --unprivileged ${targetSubnet}`, async (err) => {
      if (err) console.error('[NMAP ERROR]', err.message);

      try {
        if (fs.existsSync('/proc/net/arp')) {
          const arpData = fs.readFileSync('/proc/net/arp', 'utf8');
          const lines = arpData.split('\n');

          for (let i = 1; i < lines.length; i++) {
            const line = lines[i].trim();
            if (!line) continue;
            const parts = line.split(/\s+/);
            const ip = parts[0];
            const mac = parts[3] ? parts[3].toLowerCase() : '';

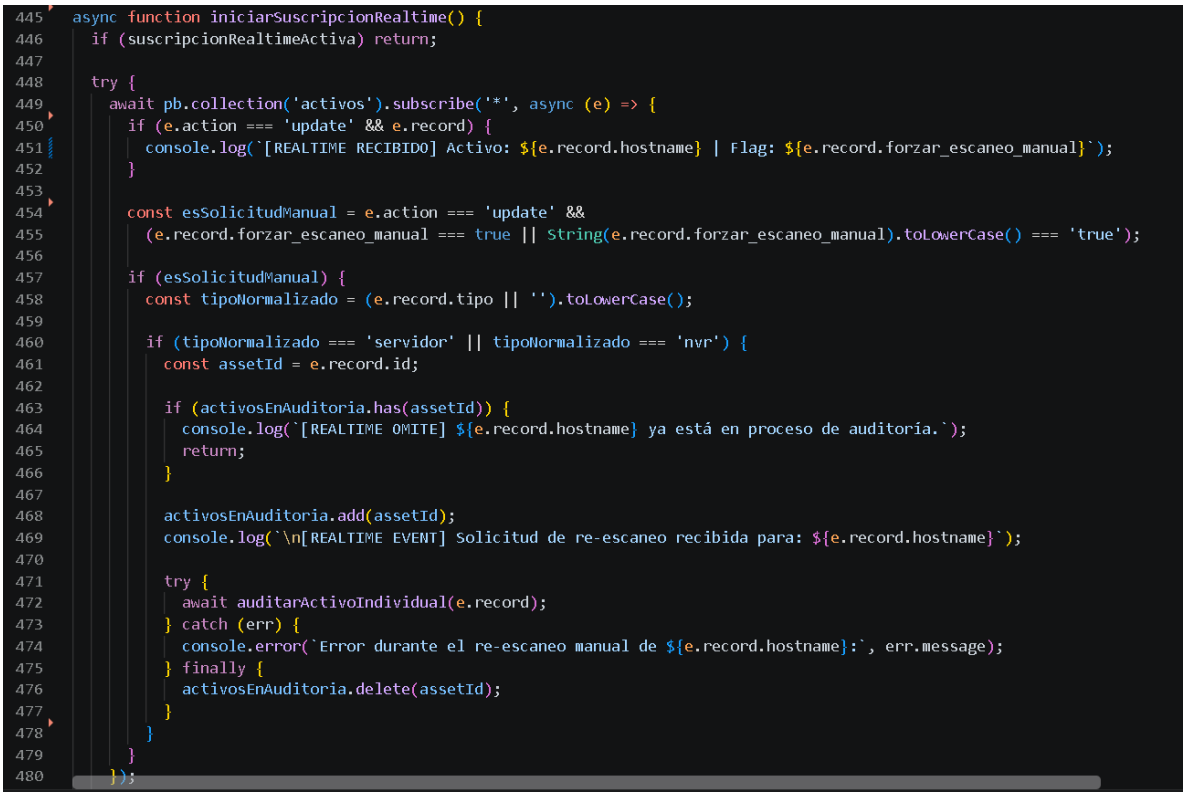
            if (ip && ip.startsWith(subnetBase + '.') && mac && mac !== '00:00:00:00:00:00' && mac.includes(':')) {
              if (!dispositivos.some(d => d.ip === ip)) {
                dispositivos.push({ ip, mac });
              }
            }
          }
        }
      } catch (arpErr) {
        console.error('[ARP ERROR]', arpErr.message);
      }
```

Worker de Auditoría de Servidores y NVRs (worker-audit.js)

Análisis de Puertos, Credenciales por Defecto y Escucha Realtime

worker-audit.js audita continuamente servidores y NVRs:

- **Detección de Contraseñas por Defecto:** Prueba credenciales típicas de fábrica (admin/admin, root/root) en interfaces web autenticadas.
- **Extracción de Firmas (Banner Grabbing):** Consulta cabeceras HTTP, certificados TLS y respuestas Nmap para identificar la versión exacta de los servicios expuestos.
- **Suscripción Realtime:** Escucha eventos de PocketBase mediante SSE. Si un administrador solicita un re-escaneo manual desde el dashboard (forzar_escaneo_manual: true), el worker intercepta la orden, ejecuta la auditoría al instante y actualiza el score de seguridad.



```
async function iniciarSuscripcionRealtime() {
  if (suscripcionRealtimeActiva) return;

  try {
    await pb.collection('activos').subscribe('*', async (e) => {
      if (e.action === 'update' && e.record) {
        console.log(`[REALTIME RECIBIDO] Activo: ${e.record.hostname} | Flag: ${e.record.forzar_escaneo_manual}`);
      }

      const esSolicitudManual = e.action === 'update' &&
        (e.record.forzar_escaneo_manual === true || String(e.record.forzar_escaneo_manual).toLowerCase() === 'true');

      if (esSolicitudManual) {
        const tipoNormalizado = (e.record.tipo || '').toLowerCase();

        if (tipoNormalizado === 'servidor' || tipoNormalizado === 'nvr') {
          const assetId = e.record.id;

          if (activosEnAuditoria.has(assetId)) {
            console.log(`[REALTIME OMITE] ${e.record.hostname} ya está en proceso de auditoría.`);
            return;
          }

          activosEnAuditoria.add(assetId);
          console.log(`\n[REALTIME EVENT] Solicitud de re-escaneo recibida para: ${e.record.hostname}`);

          try {
            await auditarActivoIndividual(e.record);
          } catch (err) {
            console.error(`Error durante el re-escaneo manual de ${e.record.hostname}:`, err.message);
          } finally {
            activosEnAuditoria.delete(assetId);
          }
        }
      }
    });
```

Módulo Agente Endpoint (Python)

Agente Local de Inspección en Endpoints (agente.py)

Inspección de Sistema Operativo, Cifrado, EDR y Canal SSE

agente.py es el ejecutable instalado en los equipos corporativos (compatible con Windows y Linux):

- **Recolección de Telemetría:** Evalúa el estado del cifrado de disco (**BitLocker** en Windows / **LUKS** en Linux), el funcionamiento del **EDR/Antivirus**, la antigüedad de los parches de seguridad y la lista de usuarios con privilegios en el grupo de administradores.
- **Cálculo de Score Local:** Pondera los controles de seguridad para asignar una puntuación de 0 a 100 y reporta las especificaciones de hardware (CPU, RAM, Disco).
- **Hilo de Escucha en Tiempo Real (SSE):** Mantiene una conexión persistente en segundo plano con la API de PocketBase para responder de inmediato a peticiones de re-escaneo forzado enviadas desde la consola de administración.

```
304  def sincronizar_activo_y_evaluacion(forzado_manual=False):
305      tipo_ciclo = "MANUAL FORZADO" if forzado_manual else "PROGRAMADO"
306      print(f"\n[CICLO {tipo_ciclo}] Iniciando recolección de métricas del sistema local...")
307
308      token = obtener_token_agente()
309      if not token:
310          print("[ERROR] No se pudo sincronizar debido a un fallo de autenticación.")
311          return
312
313      headers = {"Authorization": f"Bearer {token}"}
314      hostname = platform.node()
315      ip_local = obtener_ip_local()
316      mac_address = obtener_mac_address()
317      n_serie = obtener_numero_serie()
318
319      bitlocker_ok = verificar_bitlocker()
320      edr_ok = verificar_edr()
321      parches_ok = verificar_parches_so()
322      fecha_parche = obtener_fecha_ultimo_parche()
323      admins_lista = obtener_administradores_locales()
324      cantidad_admins = len(admins_lista)
```